

PRÁCTICA N.º 7 — HOJA DE EVIDENCIAS

Pruebas automatizadas de un microservicio

COM-600 · Microservicios · Gestión 2/2026 · Tema 5 — Desarrollo con Frameworks y Pruebas Automatizadas

Nombre y apellidos	Freddy Daniel Valencia Medina
Código universitario (CU)	35-5808
Carrera y paralelo	Ingeniería de Sistemas
Fecha de entrega	08/10/2026
Sistema operativo, versión de Node y de Docker	Linux(POP OS), node 22.23.2, Docker version 29.8.0
Repositorio (URL)	

Cómo se llena: Este documento se completa mientras se ejecuta la Parte 1 del enunciado. Cada captura y cada pregunta lleva el mismo número aquí y allá.

Las capturas deben ser de su propia máquina: debe verse el nombre de usuario o del equipo en la terminal. Recorte solo lo necesario, pero que el comando y su salida se lean con claridad.

Al terminar, exporte este archivo a PDF y súbalo a la plataforma junto con el enlace del repositorio.

Control de avance

Marque con una X a medida que complete cada laboratorio.

Laboratorio	Capturas	Preguntas	Completado
Laboratorio 0 — Verificación del entorno	1, 2	1	
Laboratorio 1 — La primera prueba unitaria	3	2	
Laboratorio 2 — Casos borde y errores: donde viven los defectos	4, 5	3	
Laboratorio 3 — Cobertura: qué mide y qué no	6, 7	4	
Laboratorio 4 — Probar el endpoint HTTP sin levantar el servidor	8, 9	—	
Laboratorio 5 — Integración real con un contenedor efímero	10, 11	5	
Laboratorio 6 — Prueba de contrato entre dos servicios	12, 13	6	
Laboratorio 7 — Dejar el proyecto listo para el pipeline	14, 15	7	

PARTE 1 — EVIDENCIAS DE LOS LABORATORIOS

Laboratorio 0 — Verificación del entorno

Captura N.º 1: salida de los cuatro comandos anteriores en su terminal.

[Pegue aquí la captura de pantalla]

Pregunta N.º 1. ¿Por qué esta práctica necesita Docker si lo que vamos a escribir son pruebas y no contenedores de la aplicación?

Captura N.º 2: salida de npm install mostrando los paquetes instalados y el archivo package.json creado.

[Pegue aquí la captura de pantalla]

Laboratorio 1 — La primera prueba unitaria

Captura N.º 3: salida de npm test con la primera prueba pasando en verde.

[Pegue aquí la captura de pantalla]

Pregunta N.º 2. El nombre de la prueba es «devuelve COMPLETADA cuando la tarea está terminada» y no «test1». ¿Qué gana el equipo cuando esa prueba falla dentro de seis meses?

Laboratorio 2 — Casos borde y errores: donde viven los defectos

Captura N.º 4: salida de npm test con las cuatro pruebas del bloque en verde.

[Pegue aquí la captura de pantalla]

Captura N.º 5: salida de la prueba en rojo, mostrando el valor esperado y el recibido.

[Pegue aquí la captura de pantalla]

Pregunta N.º 3. Una prueba que nunca ha fallado no demuestra nada. ¿Cómo puede comprobar que una prueba realmente verifica lo que dice verificar?

Laboratorio 3 — Cobertura: qué mide y qué no

Captura N.º 6: tabla de cobertura impresa en la terminal.

[Pegue aquí la captura de pantalla]

Captura N.º 7: reporte HTML de cobertura mostrando el archivo src/tarea.js con sus líneas coloreadas.

[Pegue aquí la captura de pantalla]

Pregunta N.º 4. Escriba una prueba que ejecute todas las líneas de estadoDeTarea pero que no verifique nada útil (por ejemplo, sin ningún expect que compare el resultado). ¿Qué cobertura reporta? ¿Qué conclusión saca sobre la cobertura como criterio de calidad?

Laboratorio 4 — Probar el endpoint HTTP sin levantar el servidor

Captura N.º 8: salida de npm test con las pruebas de la API en verde.

[Pegue aquí la captura de pantalla]

Captura N.º 9: salida completa de la suite con el tiempo total de ejecución visible.

[Pegue aquí la captura de pantalla]

Laboratorio 5 — Integración real con un contenedor efímero

Captura N.º 10: las dos terminales: la suite ejecutándose y docker ps mostrando el contenedor efímero levantado por la prueba.

[Pegue aquí la captura de pantalla]

Captura N.º 11: salida mostrando que el contenedor de la prueba ya no existe después de terminar la suite.

[Pegue aquí la captura de pantalla]

Pregunta N.º 5. ¿Qué problema concreto evita este enfoque frente a la alternativa de que cada estudiante instale MongoDB en su propia máquina y las pruebas apunten a esa instalación?

Laboratorio 6 — Prueba de contrato entre dos servicios

Captura N.º 12: la prueba de contrato en verde junto al resto de la suite.

[Pegue aquí la captura de pantalla]

Captura N.º 13: la prueba de contrato fallando y señalando el campo faltante.

[Pegue aquí la captura de pantalla]

Pregunta N.º 6. Las pruebas unitarias del servicio de Tareas siguieron en verde con ese cambio. ¿Por qué? ¿Qué tipo de defecto detecta la prueba de contrato que ningún otro nivel detecta?

Laboratorio 7 — Dejar el proyecto listo para el pipeline

Captura N.º 14: los dos códigos de salida: 0 con la suite en verde y distinto de 0 con la suite en rojo.

[Pegue aquí la captura de pantalla]

Pregunta N.º 7. Si una prueba es inestable (a veces pasa y a veces falla), ¿qué le ocurre al pipeline y qué le ocurre a la confianza del equipo? ¿Qué haría usted con esa prueba?

Captura N.º 15: la página del repositorio en GitHub mostrando la carpeta practica7-pruebas con tests/, contratos/ y el README.

[Pegue aquí la captura de pantalla]

Cierre de la Parte 1

Responda brevemente antes de entregar:

¿Qué parte de la práctica le resultó más difícil y por qué?

¿Qué error tuvo que resolver por su cuenta y cómo lo resolvió?

¿Qué haría distinto si tuviera que repetir esta práctica?

Declaración de autoría: Declaro que las capturas presentadas fueron tomadas en mi propio equipo y que las respuestas son de mi autoría. Entiendo que presentar evidencias de otra persona anula la Parte 1 de esta práctica.

Firma del estudiante

Fecha